

PA-MPPI: Perception-Aware Model Predictive Path Integral Control for Quadrotor Navigation in Unknown Environments

Yifan Zhai, Rudolf Reiter, and Davide Scaramuzza

Abstract—Quadrotor navigation in unknown environments is critical for practical missions such as search-and-rescue. Solving it requires addressing three key challenges: path-planning in non-convex free space due to obstacles, satisfying quadrotor-specific dynamics and objectives, and exploring unknown regions to expand the map. Recently, the Model Predictive Path Integral (MPPI) method has emerged as a promising solution that solves the first two challenges. By leveraging sampling-based optimization, it can effectively handle non-convex free space while directly optimizing over the full quadrotor dynamics, enabling the inclusion of quadrotor-specific costs such as energy consumption. However, MPPI has been limited to tracking control that only optimizes trajectories in a small neighbourhood around a reference trajectory, as it lacks the ability to explore unknown regions and plan alternative paths when blocked by large obstacles. To solve this issue, we introduce Perception-Aware MPPI (PA-MPPI). Here, perception-awareness is characterized by planning and adapting the trajectory online based on perception objectives. Specifically, when the goal is occluded, PA-MPPI's perception cost biases trajectories that can perceive unknown regions. This expands the mapped traversable space and increases the likelihood of finding alternative paths to the goal. Through hardware experiments, we demonstrate that PA-MPPI, running at 50 Hz, performs on par with the SOTA quadrotor navigation planner for unknown environments in our challenging test scenarios. In addition, we demonstrate that PA-MPPI can be used as a safe and robust action policy for navigation foundation models, which often provide goal poses that are not directly reachable.

SUPPLEMENTARY MATERIALS

Video: <https://youtu.be/DNUK2gUcVIM>

I. INTRODUCTION

ENABLING quadrotors to navigate to a goal in unknown environments autonomously is critical for practical missions such as search-and-rescue, infrastructure inspection, and exploration [1]–[3]. It is also relevant for the safe deployment of navigation foundation models, which provide navigation waypoints or goals in previously unseen environments [4], [5]. Achieving this capability requires addressing three key challenges: (i) non-convex constraints: cluttered environments

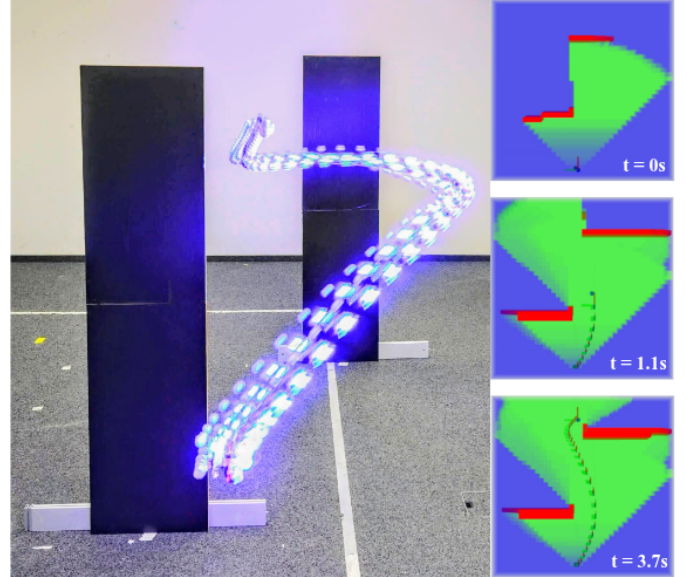


Fig. 1: The Perception-Aware MPPI (PA-MPPI) controller navigating to the goal pose while avoiding obstacles in a previously unknown environment. The controller simultaneously controls the quadrotor at 50 Hz and optimizes the perception objective based on an online-updated 3D map constructed from onboard observations.

create non-convex free space, which complicates gradient-based optimization; (ii) quadrotor-specific dynamics and costs: planned trajectories must satisfy dynamic feasibility while optimizing costs such as effort and energy; and (iii) mapping in unknown environments: since the environment must be mapped online using onboard perception, successful navigation must incorporate exploration and mapping of unknown regions to find a feasible path to the goal.

A common approach is a hierarchical planner–controller architecture [6]–[10], where a global planner computes trajectories subsequently tracked by a local controller. However, this separation often results in conservative, or dynamically infeasible plans, as quadrotor models and related costs and constraints need to be approximated [11].

Model Predictive Path Integral (MPPI) control has recently emerged as a promising alternative. By employing sampling-based optimization, MPPI can navigate non-convex and non-smooth free space in the presence of obstacles while directly optimizing control inputs on quadrotor-specific dynamics and costs [12], [13]. However, MPPI has been largely confined to following reference trajectories, demonstrating only limited local optimization near the reference, as it lacks both global

Manuscript received: September 5, 2025; Revised December 6, 2025; Accepted January 25, 2026. This paper was recommended for publication by Editor Soon-Jo Chung upon evaluation of the Associate Editor and Reviewers comments.

This work was supported by the European Union's Horizon Europe Research and Innovation Programme under grant agreement No. 101120732 (AUTOASSESS) and the European Research Council (ERC) under grant agreement No. 864042 (AGILEFLIGHT). These authors are with the Robotics and Perception Group, Department of Informatics, University of Zurich, Switzerland (<https://rpg.ifi.uzh.ch>). Contact: dzhai@ifi.uzh.ch

Digital Object Identifier (DOI): see top of this page.

map awareness and the capability to explore unknown areas and plan successful trajectories.

In this work, we introduce the Perception-Aware Model Predictive Path Integral (PA-MPPI) controller, tightly integrated with a perception and mapping module, to afford standard MPPI with the capability of navigating unknown environments without external references. This is achieved by extending standard MPPI with perception awareness, characterized by adapting control inputs and trajectory optimization based on both current and future perception of the environment [2], [14], [15]. Using the current map of the environment, PA-MPPI introduces a novel perception cost that evaluates sampled trajectories based on their potential to perceive unknown regions in the goal direction, thereby guiding the optimized trajectory to map unknown regions that lead to the goal.

We validate PA-MPPI in simulated and real-world experiments, demonstrating that perception-awareness enables MPPI to navigate through complex, unknown environments with performance comparable to SUPER [10], the current state-of-the-art planner for safety-assured quadrotor navigation in unknown environments.

Our contributions are threefold:

- **Novel Perception-Aware MPPI Formulation:** We propose a cost function that exploits the current environment map to guide trajectory optimization toward frontiers that help perceive unknown regions towards the goal, allowing MPPI to navigate without external references.
- **Integrated Framework:** We present an integrated framework that tightly couples sensing, mapping, and a high-performance MPPI implementation, running at 50 Hz for real-time quadrotor control.
- **Hardware-in-the-Loop Validation:** Our experiments show that PA-MPPI performs comparably to the current SOTA planner, SUPER [10]. We further demonstrate that using PA-MPPI as the action policy enables robust deployment of navigation foundation models.

II. RELATED WORK

Obstacle Avoidance in Cluttered Environments. In classical sense-think-act architectures, a high-level planner is responsible for more complex objectives and generates a safe trajectory, which is then passed to a fast lower-level controller [11], [16]. Often, planners operate in a discrete or low-dimensional control/state space, for example, using graph search [6]–[8] or rapidly exploring random trees [9]. However, with a reduced state or control space, trajectories are suboptimal and conservative, and the hierarchical decomposition introduces an interaction between the controller and the planner. In particular, the novel perception-awareness requires a cost associated with orientation that cannot be accounted for in classical higher-level planners, such as those in [11], [16].

Optimization-based approaches often simultaneously plan safe trajectories and control the actuators. However, optimization-based motion planning in the presence of obstacles is challenging due to the induced nonconvexity and nonsmoothness [17]–[22]. The authors in [18], [20], [21] propose efficient algorithms for ellipsoidal or cubic

obstacles, and [1] shows a real-world implementation for Unmanned Aerial Vehicles (UAVs). However, such derivative-based formulations typically rely on smooth functions, which can be difficult to obtain directly from raw depth observations in complex, highly non-smooth environments. To circumvent this, [23] generates a number of candidate reference trajectories, and assigns cost based on collision and proximity to the goal. However, the optimization that generates the candidates does not consider actuation limits, which requires additional checks and potentially multiple iterations of relaxation and re-optimization.

Beyond optimization-based approaches, learned depth-camera-based policies have shown sufficient navigation performance when trained end-to-end from visual inputs. The authors in [24] demonstrated high-speed flight in real-world environments using policies trained in simulation. More recently, vision transformer-based policies have been explored for end-to-end quadrotor obstacle avoidance using onboard computation [25]. While such approaches improve the closed-loop cost disregarding collisions, they often lack the safety-relevant explicit constraint handling and adaptability to new environments provided by optimization-based methods.

Perception-Aware Navigation. A growing number of works integrates perception objectives directly into planning and control, either to acquire task-relevant information or to maintain estimation quality. Perception-aware Model Predictive Control (MPC) aligns the robot’s viewpoint with features to maximize visibility during challenging maneuvers [2], [14], [26]. Information-theoretic surrogates such as Fisher Information Fields provide differentiable maps for actively choosing informative viewpoints [27], [28]. In exploration and inspection, planners explicitly reason about localization uncertainty to select safe, observable trajectories [3].

The research direction of our work treat unobserved space as potentially blocked or even hazardous within partial maps, thus requiring its goal-targeted exploration [6], [17], [18]. The high-level planners in [17], [18] enforce perception awareness along an optimistic trajectory by monitoring potentially hazardous regions, while [6] plans to explore the frontier of unknown space via a Dijkstra graph-search. The current state-of-the-art planner, SUPER [10], simultaneously plans an exploratory trajectory, guided by A* graph-search in both free and unknown space for exploration and high speed flight, and a backup trajectory in known free space for emergency stops. As these ideas typically resort to a separate planner, they motivate our use of perception-driven costs in a single controller that rewards the line of sight to the goal and prevents motion into unknown regions, with a focus on highly cluttered environments obtained from depth images.

Algorithms for Model Predictive Control. In the following, the proposed perception-aware Model Predictive Path Integral (MPPI) algorithm is motivated, which extends classical MPPI [29] by a perception cost.

a) *Derivative-based MPC.*: When objectives and constraints admit smooth approximations, derivative-based MPC offers strong local convergence and tight constraint handling. Progressive smoothing and continuation strategies have been proposed to navigate nonconvex obstacle costs [21], con-

ceptually paralleling equal to annealing in sampling-based MPC [30]. A recent approach proposed an algorithm with an external active set solver for cluttered point-cloud obstacles [31], which achieves a feasible average but has an ample worst-case computation time. Still, in settings where, in addition to obstacles represented by raw depth maps, the cost functions are also highly nonsmooth, constructing reliable differentiable surrogates remains challenging. Sampling methods showed superior performance in these settings [32].

b) Sampling-based MPC.: Sampling-based MPC methods, such as MPPI, optimize control sequences by Monte Carlo rollouts instead of using local gradients, making them attractive for nonconvex, nonsmooth objectives and dynamics. The most straightforward but largely inefficient random shooting method purely randomizes actions [33]. The Cross Entropy Method (CEM) [34] iteratively refines a distribution toward high-performing controls via an elite-only update and has been widely adopted as a trajectory optimizer and within model-based RL pipelines [35]. MPPI control [29], [36], [37] uses all samples via weighting to iteratively refine a trajectory [38]. MPPI has recently been pushed to demanding robot platforms, including agile UAVs [12] and whole-body locomotion [30], [39]. However, these works rely on external references, such as a reference quadrotor trajectory or quadruped walking gait joint position reference. PA-MPPI, on the other hand, enables reference-free navigation via the novel perception-aware cost.

III. PRELIMINARIES

A. Notation

We introduce two reference frames: W , the fixed world frame, whose z -axis is gravity-aligned, and B , the quadrotor body frame, whose x -axis aligns with the onboard camera's principal axis. In this paper, vectors and matrices are written in bold, with matrices indicated by capital letters. Each vector carries a subscript specifying the frame in which it is expressed and its endpoint. For instance, $\mathbf{p}_{WB} \in \mathbb{R}^3$ denotes the position of the body frame B relative to the world frame W , and $\mathbf{R}_{WB}$ denotes the rotation from frame B to W . We use $\mathbf{y}_{1:N} \in \mathbb{R}^{N \times n_y}$ for the vectorization of vectors $\mathbf{y}_1, \dots, \mathbf{y}_N$, with $\mathbf{y}_i \in \mathbb{R}^{n_y}$. The quaternion algebra is $\mathbb{H}$ and $q \in \mathbb{H}_1 := \{q \in \mathbb{H} \mid \|q\| = 1\}$.

B. Model Predictive Control

Given an environment, often formulated as a Markov decision process (MDP), with the states $\mathbf{x} \in \mathbb{R}^{n_x}$, controls $\mathbf{u} \in \mathbb{R}^{n_u}$, a stage cost $l : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \rightarrow \mathbb{R} \cup \{\infty\}$, a discount factor γ , and a stochastic model $P : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \rightarrow \text{Dist}(\mathbb{R}^{n_x})$, MPC provides a means to yield nearly optimal (sufficiently suboptimal) controls by solving an approximated implicit version of the MDP online, local at the current state $\hat{\mathbf{x}} \in \mathbb{R}^{n_x}$, cf. [40] for details. MPC typically uses a simplified deterministic model $f : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \rightarrow \mathbb{R}^{n_x}$, possibly simplified stage cost $\ell : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \rightarrow \mathbb{R}$ and an approximation $\tilde{V} : \mathbb{R}^{n_x} \rightarrow \mathbb{R}$ of the optimal value function $V : \mathbb{R}^{n_x} \rightarrow \mathbb{R}$. Moreover, an open-loop trajectory $\mathbf{u}_{0:H-1}$ with open-loop states $\mathbf{x}_i \in \mathbb{R}^{n_x}$, actions $\mathbf{u}_i \in \mathbb{R}^{n_u}$ and horizon H is optimized, instead of

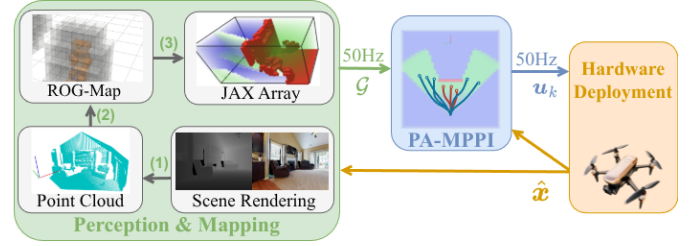


Fig. 2: Illustration of the control stack. The perception and mapping module has three parts: (1) a point cloud is generated from the rendered depth image and transformed to the world frame, (2) the ROG-Map [42] updates the occupancy map with the new point cloud, and (3) the ROG-Map is converted to a JAX array as input to PA-MPPI.

a policy. The resulting single shooting MPC optimization problem is therefore

$$\min_{\mathbf{u}} \tilde{V}(\mathbf{x}_H; \mathbf{p}) + \sum_{k=0}^{H-1} \ell(\mathbf{x}_k, \mathbf{u}_k; \mathbf{p}) \quad (1)$$

with $\mathbf{x}_0 = s$, $\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k)$, $0 \leq k < H$,

where constraints are approximated in the value and cost function. We denote the dependency of the cost function on external parameters, such as obstacle parameters or the goal by $\mathbf{p} \in \mathbb{R}^{n_p}$. Unlike gradient-based algorithms for solving the MPC problem (1), which typically rely on first or second-order derivatives, MPPI uses Monte Carlo sampling-based optimization. Despite the initial derivation of MPPI control for the stochastic system [29], [36], [37], it is often used for the deterministic counterpart (1), where noise is added as part of the optimization algorithm [41].

IV. METHODOLOGY

A graphical overview of the integrated control stack is shown in Fig. 2. As illustrated, PA-MPPI receives an occupancy grid from the perception module and optimizes a trajectory as control sequences, which are directly executed by the quadrotor. The proposed PA-MPPI algorithm utilizes a quadrotor dynamics model, as described in Sect. IV-A. A detailed description of the MPPI formulation is given in Sect. IV-B. The perception and mapping part, and the cost definition are detailed in Sect. IV-C and IV-D, respectively.

A. Quadrotor Dynamics

In addition to the quadrotor's position $\mathbf{p}_{WB}$, we define orientation, and linear velocity in the world frame by $\mathbf{q}_{WB} \in \mathbb{H}_1$, and $\mathbf{v}_{WB} \in \mathbb{R}^3$, respectively, and the quadrotor's angular velocity in the body frame by $\boldsymbol{\omega}_B \in \mathbb{R}^3$. The collective thrust and the corresponding thrust vector in the body frame are defined as $c = c_1 + \dots + c_{N_{\text{rot}}}$ and $\mathbf{c}_B = [0 \ 0 \ c]^\top$, where c_i is the thrust generated by the i -th of N_{rot} motors. The quadrotor mass is m and $\mathbf{g}_W$ is the gravity vector in the world frame. Finally, the diagonal moment of inertia matrix is $\mathbf{J} \in \mathbb{R}^{3 \times 3}$, and the body torque is $\boldsymbol{\tau}_B \in \mathbb{R}^3$. The quadrotor dynamics can then be expressed as

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\mathbf{p}}_{WB} \\ \dot{\mathbf{q}}_{WB} \\ \dot{\mathbf{v}}_{WB} \\ \dot{\boldsymbol{\omega}}_B \end{bmatrix} = \begin{bmatrix} \mathbf{v}_{WB} \\ \frac{1}{2} \Lambda(\boldsymbol{\omega}_B) \cdot \mathbf{q}_{WB} \\ \mathbf{q}_{WB} \odot \mathbf{c}_B / m + \mathbf{g}_W \\ \mathbf{J}^{-1}(\boldsymbol{\tau}_B - \boldsymbol{\omega}_B \times \mathbf{J} \cdot \boldsymbol{\omega}_B) \end{bmatrix}. \quad (2)$$

The lowest-level flight controller tracks the zero-order hold PA-MPPI control $\mathbf{u}_t = [c_t \ \boldsymbol{\omega}_{B,t}]^\top$. To ensure a feasible total thrust and body rate at each timestep, we follow the single motor thrust clipping in [12], using the motor thrust limits to acquire clipped control input $\mathbf{u}_t^{\text{clip}}$, which is then used by PA-MPPI to simulate the dynamics (2) via forward Euler integration.

B. MPPI formulation

In the MPPI framework, at each timestep k , N parallel trajectories of H steps are sampled by adding multivariate Gaussian noise to the nominal control input $\mathbf{u}_{k:k+H}^{\text{nom}}$. The perturbed control sequences $\mathbf{u}_{k:k+H}^j$, $j = 1, \dots, N$, are then rolled out from the quadrotor state $\mathbf{x}_k$ using the model (2). Each sampled trajectory $\mathbf{x}_{k:k+H}^j$ is then evaluated by summing the per-step costs $\mathcal{L}^j = \sum_{i=k}^{k+H-1} \ell(\mathbf{x}_i^j, \mathbf{u}_i^j) + \bar{V}(\mathbf{x}_H^j)$. The optimized control sequence is calculated using the exponentially weighted average based on the summed cost

$$\mathbf{u}_{k:k+H-1} = \sum_{j=1}^N w^j \mathbf{u}_{k:k+H-1}^j, \quad w^j = \frac{\exp(-\frac{\mathcal{L}^j - \mathcal{L}^{\min}}{\lambda})}{\sum_{j=1}^N \exp(-\frac{\mathcal{L}^j - \mathcal{L}^{\min}}{\lambda})},$$

where $\mathcal{L}^{\min}$ is the lowest summed cost out of the N rollouts, and λ is the temperature parameter. A low λ assigns higher weight to the best-performing rollout, while higher values assign more uniform weights to all rollouts [38]. The first action of the averaged sequence is then executed, and the optimization process repeats in the next time step.

Recent works [12], [30], [43] have shown success in decoupling the prediction time step size Δt_{pred} , and the control step size Δt_{ctrl} . With $\Delta t_{\text{pred}} > \Delta t_{\text{ctrl}}$, the policy rollouts can predict over a longer real-time horizon for the same number of forward simulation steps, allowing optimization of actions further into the future. Since the optimization loop executes the first action at control frequency and the actions in the sequence are spaced by Δt_{pred} , to reuse the remaining actions as $\mathbf{u}^{\text{nom}}$ for the next iteration, the control sequence is linearly interpolated and shifted by Δt_{ctrl} , then down-sampled at Δt_{pred} .

While the terminal value $\bar{V}$ function should in principle resemble the optimal value function and account for a recursive feasible safe set, cf. [40], we only consider a simple terminal hovering safe set with zero velocity and resort to a long enough planning horizon to diminish its influence on the open loop cost.

C. Perception & Environment Mapping

We use a depth sensor on the quadrotor to continuously build a 3D map of the environment during navigation. From each depth image, we reproject a point cloud into the world frame using the camera pose. The point clouds are then inserted into a ROG-Map [42], which efficiently aggregates all past observations into a 3D occupancy grid representation, in which each voxel has one of three states: occupied, free, or unknown, with integer values $\{1, 0, -1\}$ respectively. Formally, the 3D occupancy grid is defined as $\mathcal{G} \in \{-1, 0, 1\}^{X \times Y \times Z}$. We denote by $\mathcal{G}(\mathbf{p}_{WB})$ the operation that looks up the voxel corresponding to position $\mathbf{p}_{WB}$ and returns its value. The mapping pipeline is capable of processing depth images and

updating the occupancy grid at 50Hz for a $5 \times 5 \times 2\text{m}$ grid with a voxel resolution of 0.1 m.

D. Optimization Cost Definition

At each timestep k within the planning horizon H , $k = 0, \dots, H-1$, the stage cost of (1) for the navigation task consists of the following terms:

$$\begin{aligned} \ell_{\text{goal}} &= -c_{\text{goal}} \cdot \max(0.0, \mathbf{d}_0 - \mathbf{d}_k) \\ \ell_{\text{goal}, H-1} &= -c_{\text{goal}, H-1} \cdot \max(0.0, \mathbf{d}_0 - \mathbf{d}_{H-1}) \\ \ell_{\text{act}} &= \|\mathbf{u}\|_R^2 + \|\Delta \mathbf{u}\|_{R_\Delta}^2, \\ \ell_{\text{vel}} &= \exp(-c_{\text{vel}} \cdot \mathbf{d}_k^2) \cdot \|\mathbf{v}_k\|^2 \\ \ell_{\text{progress}} &= -c_{\text{progress}} \cdot \|\mathbf{p}_{WB,i} - \mathbf{p}_{WB,i+1}\| \\ \ell_{\text{collision}} &= c_{\text{collision}} \cdot \mathbb{1}_{\{\mathcal{G}(\mathbf{p}_{WB}) \neq 0\}}, \\ \ell_{\text{perception}} &= c_{\text{PoI}} \cdot (1 - \langle \hat{\mathbf{x}}_{WB}, \hat{\mathbf{d}}_{\text{goal}} \rangle)^2 \cdot \mathbb{1}_{\{\|\mathbf{d}_{\text{goal}}\| > c_{\text{thresh}}\}} \\ &\quad + c_{\text{occupied}} \cdot \mathbb{1}_{\{\mathcal{G}(\mathbf{r}(t^*)) = 1\}} \\ &\quad + c_{\text{unknown}} \cdot \mathbb{1}_{\{\mathcal{G}(\mathbf{r}(t^*)) = -1\}} \end{aligned}$$

and the quantities:

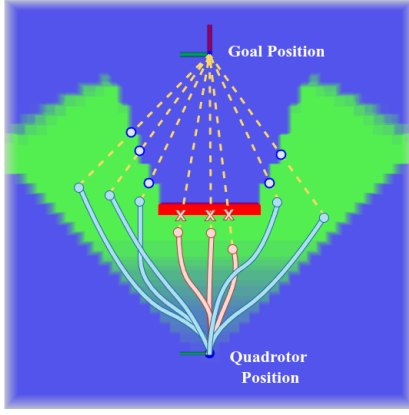
$$\begin{aligned} \mathbf{d}_i &= \|\mathbf{p}_{WB,i} - \mathbf{p}_{\text{goal}}\|, \quad \hat{\mathbf{d}}_{\text{goal}} = \frac{\mathbf{d}_{\text{goal}}}{\|\mathbf{d}_{\text{goal}}\|}, \\ \hat{\mathbf{x}}_{WB} &= \mathbf{R}_{WB} \cdot \mathbf{e}_1. \end{aligned}$$

The terminal value function is $\bar{V}(\mathbf{x}) = c_{\text{safe}} \mathbb{1}_{\{\|\mathbf{v}_{WB}\| > \underline{\mathbf{v}}\} \vee \|\boldsymbol{\omega}_B\| > \underline{\boldsymbol{\omega}}\}}$, with bounds $\underline{\mathbf{v}}$ and $\underline{\boldsymbol{\omega}}$, and a large penalty c_{safe} to account for safe set (hovering).

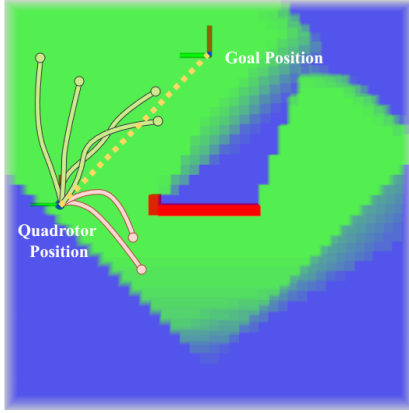
The stage costs have two distinct phases: when the goal is not within the direct line of sight of the quadrotor, as shown in Fig. 3a, to encourage exploration, the cost is $\ell = \ell_{\text{goal}} + \ell_{\text{goal}, H-1} + \ell_{\text{act}} + \ell_{\text{collision}} + \ell_{\text{perception}}$, with $c_{\text{goal}} = 0.125$ and $c_{\text{goal}, H-1} = 10$; when the goal is within the direct line of sight, as shown in Fig. 3b, to quickly reach the goal and hover, the cost is $\ell = \ell_{\text{goal}} + \ell_{\text{act}} + \ell_{\text{collision}} + \ell_{\text{progress}} + \ell_{\text{vel}}$, with $c_{\text{goal}} = 5.0$. The goal cost ℓ_{goal} encourages getting closer to the goal compared to the starting position of the current horizon. For the last step of the horizon $k = H-1$, an additional cost $\ell_{\text{goal}, H-1}$ is given to weight the end position of the horizon more, similar to the intermediate point cost in [23]. This prevents greedy behavior and allows the policy to detour and go around obstacles if that brings the policy closer to the goal in the end. The action cost ℓ_{act} penalizes the magnitude and change in control inputs, similar to the implementation in [12]. The velocity penalty ℓ_{vel} encourages the quadrotor to slow down near the goal, and the progress cost ℓ_{progress} encourages fast movement when a straight line path to the goal is available.

The collision cost $\ell_{\text{collision}}$ is a binary value weighted by a large constant $c_{\text{collision}}$. The indicator function $\mathbb{1}_{\{\mathcal{G}(\mathbf{p}_{WB}) \neq 0\}}$ returns 1 if the quadrotor's current position lies outside the set of free voxels defined in Section IV-C. This large penalty enforces the quadrotor not only to avoid collisions with known obstacles but also to refrain from entering unknown regions, which is critical for ensuring safety when navigating in unknown environments.

The perception cost, $\ell_{\text{perception}}$, consists of two components. The first, weighted by c_{PoI} , encourages alignment of the quadrotor's x-axis (coinciding with the depth camera's



(a) Given only one depth observation, there is no position in known free space that has a direct line of sight to the goal. Trajectories receive a reward for exploring unknown regions (blue trajectories) or a penalty for facing obstacles (red trajectories).



(b) After moving to a new position while mapping the environment, there is a direct line of sight from the current position to the goal, and the second-phase cost is active, encouraging the quadrotor to directly reach the goal, and the perception cost is not assigned anymore.

Fig. 3: A top-down visualization of the ray-tracing in perception cost calculation, showing the occupied voxels (red), free voxels (green), and unknown voxels (blue), and sampled trajectories on which ray-tracing is performed.

principal axis) with the direction of the goal position (point of interest), thereby maximizing the goal's visibility within the image frame [2], [14]. As the quadrotor approaches the goal (distance below c_{thresh}), this term becomes inactive. The last two terms in $\ell_{\text{perception}}$ represent two mutually exclusive cases of the quadrotor position with respect to the mapped region of the environment. We define a ray that starts from the quadrotor position $\mathbf{p}_{WB}$ and ends at the goal position $\mathbf{p}_{\text{goal}}$ as: $\mathbf{r}(t) = \mathbf{p}_{WB} + t \cdot \mathbf{d}_{\text{goal}}$, $0 \leq t \leq 1$. Since $\mathbf{p}_{WB}$ is constrained to the known free space due to the collision cost, there exists a t^* such that the ray either exits the free space at an obstacle or unknown space, as shown in Fig. 3a. If $\mathbf{r}(t^*)$ lies in an occupied voxel, a cost c_{occupied} is assigned. If $\mathbf{r}(t^*)$ falls in an unknown voxel, then it suggests an exploration frontier towards the goal is present, and a negative cost c_{unknown} is given. When there is a direct line of sight from the current position to the goal, the second-phase cost is active, as there is no need for exploration, and no perception cost is assigned. As illustrated in Fig. 3, this ray-tracing term favors sampled trajectories that either explore unknown regions when the goal is blocked by obstacles (Fig. 3a) or move directly toward the goal when possible, cf. Fig. 3b. This design enables

TABLE I: PA-MPPI parameters.

MPPI param.		Cost definition constants			
N	17,500	c_{pos}	2.5	c_{vel}	5.0
H	15	c_{ψ}	1.0	c_{unknown}	-4.0
λ	0.02	$c_{\text{collision}}$	15.0	c_{occupied}	2.0
Δt_{pred}	0.1 s	c_{Pol}	5.0	R	diag(0.01, 0.025, 0.025, 0.2)
Δt_{ctrl}	0.02 s	c_{thresh}	0.5	R_{Δ}	diag(0.02, 0.05, 0.05, 0.05)

the PA-MPPI controller to exploit map information, allowing it to plan around obstacles and efficiently explore unknown space. For the implementation of ray tracing, we adopt the 3D Digital Differential Analyzer (DDA) algorithm [44], which does not require a signed distance field representation of the environment. Due to the high computational cost, ray tracing is performed at the last step, $k = H - 1$, of the open-loop trajectory.

V. EXPERIMENTS

To evaluate the performance of PA-MPPI without considering the effect of other modules in the control loop, such as depth sensor noise or imperfect state estimation, we conduct simulated and hardware-in-the-loop (HIL) [45] experiments. In the HIL setting, we use a motion capture system to acquire the ground truth state of the quadrotor, and the Flightmare simulator [46] to render depth images in real time. The PA-MPPI controller is implemented in JAX and integrated into the Agilicious control framework [45], running on a laptop with an i7-13800H CPU with 64GB RAM and a NVIDIA A1000 laptop GPU with 6GB VRAM. The quadrotor used has a mass of 0.21 kg and arm length $l = 19.4$ cm, with propeller radius of 3.81 cm and a thrust to weight ratio of 6.8.

Two sets of experiments are conducted. The first consists of navigating synthetic scenes of varying difficulty (Fig. 4) to quantitatively evaluate the performance of PA-MPPI. Robustness of PA-MPPI is also investigated by injecting wind disturbances during navigation. The second consists of indoor navigation scenes from the Habitat Matterport dataset [47], using goal poses proposed by a navigation foundation model [4] to demonstrate an example usage of PA-MPPI as the action policy for a Vision-Language-Action (VLA) model in unknown environments. A list of PA-MPPI parameters are provided in Table I. As in our experiments, the weight of the terminal safe set c_{safe} barely had an influence, we set it to zero.

Synthetic Scene Experiment. To quantitatively evaluate the performance of PA-MPPI, we design three scenes for navigation. The first is a C-shaped wall of varying sizes, a challenging obstacle against greedy policies. The second is a hole in a wall, similar to manholes that quadrotors must navigate through during ship inspections [48]. The third scene consists of four walls that the quadrotor must navigate past, which tests path-finding capabilities and precise control through narrow gaps of 0.5 m. The difficulty of each scene is parametrized by the size of the obstacles w (each difficulty tested 5 times) or the diameter of the holes r (5 random locations, each tested 2 times), as shown in Fig. 4. We validate the sufficient difficulty of the scenes by first showing that a gradient-based trajectory optimization algorithm, EGO-Planner [22], fails to solve the tasks at harder difficulties. As shown in Fig. 5,

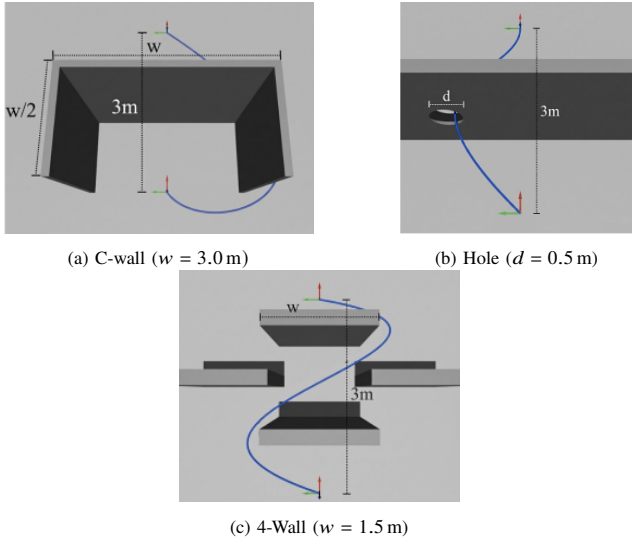


Fig. 4: Synthetic scenes for navigation experiments. The goal pose for each task is always 3m ahead of the initial pose, with three types of obstacles in between: a C-shaped wall (a), a wall with a hole (b), and four walls (c). The most challenging setting for each scene is illustrated here, with example successful trajectories depicted in blue.

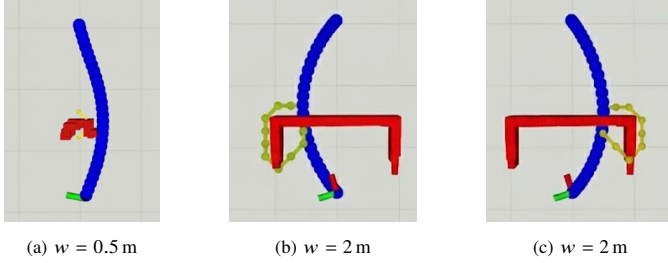


Fig. 5: Although EGO-Planner [22] is able to optimize the trajectory to navigate past the C-shaped wall in the easy setting (a), it struggles to find a feasible trajectory in the harder settings (b)(c). Yellow/Green: A* obstacle avoidance proposals, Blue: planned trajectories.

while EGO-Planner is able to successfully navigate past a small obstacle, the optimization fails to converge when the A* obstacle avoidance front-end deviates significantly from the initial trajectories, resulting in huge feasibility costs that prevent the optimization from converging.

As EGO-Planner [22] and trajectory tracking MPPI [12] cannot navigate around obstacles beyond the easiest settings, we use SUPER [10], the state-of-the-art safety-assured MAV navigation planner in unknown environments, as the baseline. SUPER replans at 10 Hz, and a geometric controller is implemented to track the trajectories. We tune SUPER's parameter to be as fast as possible without collisions in our test scenes.

The experiment results are summarized in Table II, III, IV. Guided by the perception cost, PA-MPPI moves to positions that help navigate around obstacles and map the environment towards the goal, successfully completing the tasks, as visualized in Fig. 6 and 7. While both SUPER and PA-MPPI are able to complete the tasks with 100% success rate, PA-MPPI consistently performs better in terms of total time and velocity of the trajectories, while having trajectory distances on par with SUPER. For a qualitative comparison, we visualize two trajectories from SUPER in Fig. 8. For the C-wall scene, although SUPER's A* front-end proposed trajectories close to the obstacle wall, the subsequent trajectory optimization

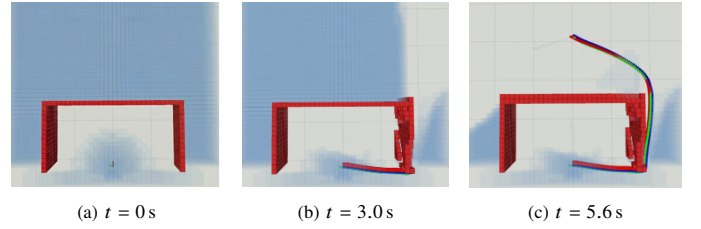


Fig. 6: PA-MPPI trajectory (C-wall with $w = 3.0$ m), total distance 5.7 m.

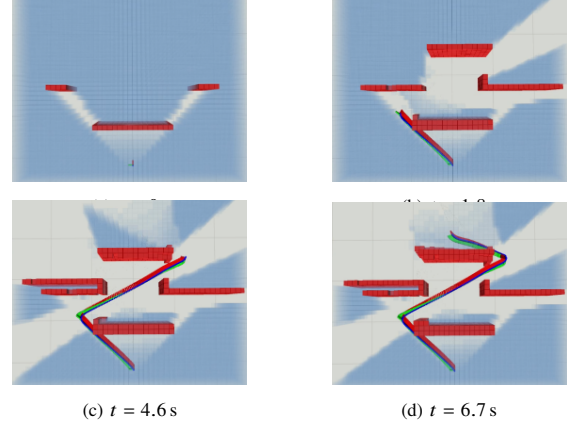


Fig. 7: PA-MPPI trajectory (4-Wall with $w = 1.5$ m), total distance 6.3 m

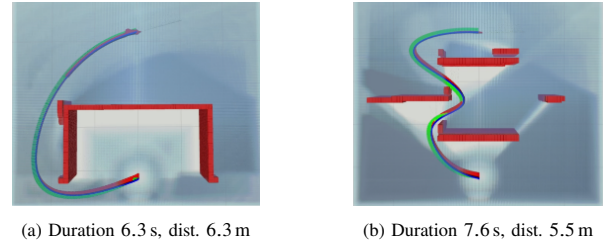


Fig. 8: SUPER trajectory samples in the C-wall and the 4-wall scene

TABLE II: C-Shaped Wall Experiment Results.

	Obs. Size w	Dist. (m)	Time (s)	Avg. Vel. (m s^{-1})	Energy Consumption (J)
SUPER (sim)	1.0 m	3.57 ± 0.15	4.54 ± 0.51	0.79 ± 0.05	217.1 ± 25.3
	2.0 m	4.54 ± 0.12	4.72 ± 0.28	0.96 ± 0.03	225.0 ± 13.2
	3.0 m	6.48 ± 0.92	6.00 ± 0.84	1.08 ± 0.03	286.8 ± 42.0
PA-MPPI (sim)	1.0 m	3.56 ± 0.04	2.91 ± 0.04	1.22 ± 0.03	142.0 ± 2.0
	2.0 m	4.56 ± 0.07	4.20 ± 0.07	1.09 ± 0.01	202.6 ± 3.5
	3.0 m	5.87 ± 0.07	5.14 ± 0.22	1.14 ± 0.06	249.5 ± 9.5
PA-MPPI (real)	1.0 m	3.61 ± 0.13	2.84 ± 0.05	1.27 ± 0.04	—
	2.0 m	4.59 ± 0.10	4.09 ± 0.09	1.12 ± 0.02	—
	3.0 m	5.88 ± 0.11	5.13 ± 0.18	1.15 ± 0.06	—

aggressively optimized for dynamic feasibility by penalizing jerk, body rate, and collective thrust, resulting in a smoother but longer trajectory, as shown in Fig. 8a. In the 4-wall scene, the trajectory adheres to the shortest path in distance suggested by the A* front end. However, compared to PA-MPPI's trajectory, the final trajectory requires significantly more rotational control effort, as A* is unaware of the quadrotor dynamics. On the other hand, PA-MPPI inherently samples dynamically feasible trajectories, allowing it to prioritize trajectory quality in the sampling-based optimization. As a result, PA-MPPI's trajectories are faster and significantly more energy-efficient.

TABLE III: Hole in the Wall Experiment Results.

	Hole Diameter d	Dist. (m)	Time (s)	Avg. Vel. (m s^{-1})	Energy Consumption (J)
SUPER (sim)	0.5 m	3.77 ± 0.41	4.36 ± 0.44	0.87 ± 0.05	209.0 ± 21.8
	1.0 m	3.83 ± 0.79	4.26 ± 0.56	0.89 ± 0.08	203.4 ± 27.3
PA-MPPI (sim)	0.5 m	3.66 ± 0.40	3.23 ± 0.42	1.14 ± 0.07	155.7 ± 20.3
	1.0 m	3.44 ± 0.38	2.79 ± 0.35	1.24 ± 0.08	135.8 ± 17.1
PA-MPPI (real)	0.5 m	3.71 ± 0.45	3.22 ± 0.65	1.17 ± 0.11	—
	1.0 m	3.52 ± 0.37	2.67 ± 0.30	1.32 ± 0.10	—

TABLE IV: 4-Wall Experiment Results.

	Obs. Size w	Dist. (m)	Time (s)	Avg. Vel. (m s^{-1})	Energy Consumption (J)
SUPER (sim)	0.5 m	3.42 ± 0.14	4.30 ± 0.14	0.79 ± 0.02	204.7 ± 6.1
	1.0 m	5.22 ± 0.60	6.53 ± 0.34	0.80 ± 0.08	313.8 ± 17.0
	1.5 m	5.47 ± 1.25	7.57 ± 0.99	0.72 ± 0.07	364.3 ± 50.0
PA-MPPI (sim)	0.5 m	3.78 ± 0.20	3.12 ± 0.08	1.19 ± 0.05	155.1 ± 4.1
	1.0 m	3.97 ± 0.12	4.52 ± 0.15	0.88 ± 0.03	216.7 ± 7.0
	1.5 m	6.03 ± 0.33	5.78 ± 0.23	1.05 ± 0.07	281.7 ± 11.2
PA-MPPI (real)	0.5 m	3.69 ± 0.09	2.94 ± 0.13	1.26 ± 0.05	—
	1.0 m	3.95 ± 0.07	4.03 ± 0.10	0.98 ± 0.04	—
	1.5 m	5.89 ± 0.32	6.75 ± 0.75	0.88 ± 0.10	—

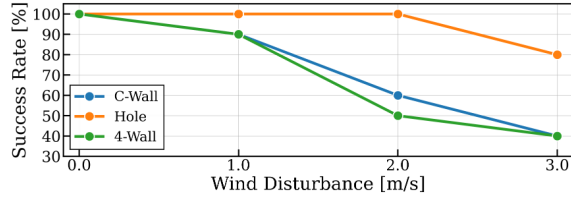


Fig. 9: Success rate of PA-MPPI under wind disturbance

Robustness Study. To investigate the robustness of PA-MPPI, we conduct simulated test in the most challenging setting of the three tasks while the quadrotor is subjected to external wind disturbances. The wind directions are randomly sampled in the xy -plane for each episode, with wind magnitudes of $[1.0 \text{ m s}^{-1}, 2.0 \text{ m s}^{-1}, 3.0 \text{ m s}^{-1}]$ with 10 test episodes each. The results are shown in Fig. 9. With low external wind speed, PA-MPPI retains a high success rate, while at larger wind speeds, the controller cannot compensate for the disturbance, resulting in more episodes failing by colliding into the obstacles or being blown into unknown areas, resulting in early terminations.

PA-MPPI combined with Navigation Foundation Model.

We validate the real-world applicability of the PA-MPPI controller by using it as the action policy for a navigation foundation model, NoMaD [4], which proposes trajectory waypoints. Since the inputs to NoMaD are only monocular RGB images, the proposed waypoints are not guaranteed to be feasible due to scale ambiguity. We use two scenes from the Habitat Matterport dataset [47]. In the first scene, visualized in Fig. 10a, the NoMaD proposed trajectory attempts to enter the room but misses the door. PA-MPPI was able to explore and map unknown regions in this scene, navigate through the door, and ultimately reach the goal position, as shown in Fig. 10b. In the second scene, visualized in Fig. 10c, the NoMaD trajectory starts close to a ping-pong table and goes directly through it. PA-MPPI successfully avoids the obstacle. Both scenarios demonstrate successful reference-free navigation in

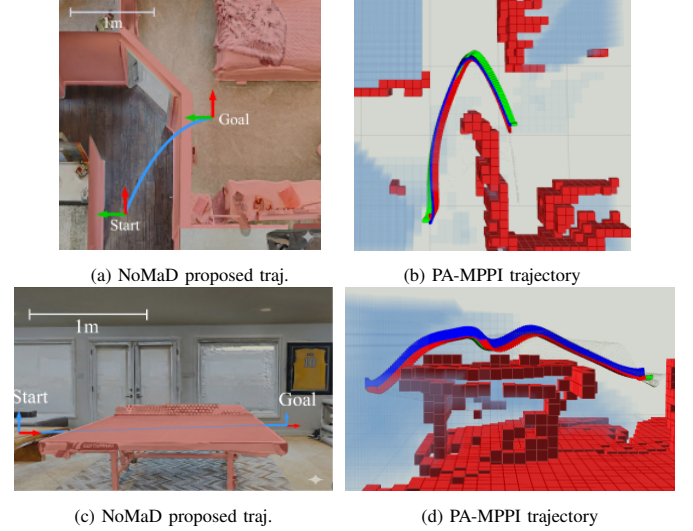


Fig. 10: Two Habitat scenes, with obstacles (walls, furniture, etc) overlaid in red and NoMaD proposed trajectory in blue (a)(c), and visualization of the PA-MPPI trajectory in the occupancy map (b)(d).

cluttered environments using PA-MPPI, making it a suitable action policy for navigation foundation models.

VI. DISCUSSION & FUTURE WORK

Due to the limited FoV of the depth sensor, a good initial observation is crucial for successful navigation around large obstacles. For example, in the experiment shown in Fig. 6, the initial observation at $t = 0 \text{ s}$ is initialized by turning the quadrotor $\pm 90^\circ$ to observe free space outside the convex hull of the C-shaped wall. However, this issue can be solved by replacing the depth camera with LIDAR, which only requires modification to the mapping module. Additionally, we only considered navigation tasks within a fixed 3D boundary. Future work may take advantage of the local map sliding feature of the ROG-Map to implement larger scale navigation. Finally, PA-MPPI's planning horizon is constrained by available computation, which fundamentally limits the complexity of navigation scenes it can handle. This may be mitigated by using an appropriate terminal value function on the last position of the horizon. We leave this for future work to investigate.

VII. CONCLUSION

This work presents a perception-aware MPPI controller that integrates a novel perception-driven cost to enable reference-free quadrotor navigation in partially known, cluttered environments with MPPI. By exploiting the current map, the perception term steers trajectories toward informative frontiers to explore the unknown regions and advance towards the goal. Simulated and real-world experiments demonstrate that its performance is comparable to that of the state-of-the-art navigation planner. We further demonstrate its potential as an action policy for foundation models to navigate challenging environments. Future work will focus on extending PA-MPPI to longer-horizon navigation and conducting an in-depth study of its path-planning limits and methods to mitigate them.

REFERENCES

- [1] L. F. Recalde, B. S. Guevara, C. P. Carvajal, V. H. Andaluz, J. Varela-Aldás, and D. C. Gandolfo, "System identification and nonlinear model predictive control with collision avoidance applied in hexacopters uavs," *Sensors*, vol. 22, no. 13, 2022.
- [2] J. Xing, G. Cioffi, J. Hidalgo-Carrió, and D. Scaramuzza, "Autonomous power line inspection with drones via perception-aware MPC," in *IROS*, 2023.
- [3] C. Papachristos, F. Mascarich, S. Khattak, T. Dang, and K. Alexis, "Localization uncertainty-aware autonomous exploration and mapping with aerial robots using receding horizon path-planning," *Autonomous Robots*, vol. 43, no. 8, pp. 2131–2161, 2019.
- [4] A. Sridhar, D. Shah, C. Glossop, and S. Levine, "Nomad: Goal masked diffusion policies for navigation and exploration," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 63–70.
- [5] A.-C. Cheng, Y. Ji, Z. Yang, X. Zou, J. Kautz, E. Biyik, H. Yin, S. Liu, and X. Wang, "Navila: Legged robot vision-language-action model for navigation," in *RSS*, 2025.
- [6] T. Cieslewski, E. Kaufmann, and D. Scaramuzza, "Rapid exploration with multi-rotors: A frontier selection method for high speed flight," in *IROS*, 2017.
- [7] M. Naazare, D. Ramos, J. Wildt, and D. Schulz, "Application of graph-based path planning for uavs to avoid restricted areas," in *IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, 2019.
- [8] W. Liu, Y. Ren, and F. Zhang, "Integrated planning and control for quadrotor navigation in presence of suddenly appearing objects and disturbances," *IEEE Robotics and Automation Letters*, 2024.
- [9] M. W. Achtelik, S. Weiss, M. Chli, and R. Siegwart, "Path planning for motion dependent state estimation on micro aerial vehicles," in *IEEE International Conference on Robotics and Automation*, 2013.
- [10] Y. Ren, F. Zhu, G. Lu, Y. Cai, L. Yin, F. Kong, J. Lin, N. Chen, and F. Zhang, "Safety-assured high-speed navigation for mavs," *Science Robotics*, vol. 10, no. 98, p. ead06187, 2025.
- [11] M. Hehn and R. D'Andrea, "Real-time trajectory generation for quadcopters," *IEEE Transactions on Robotics*, 2015.
- [12] M. Minařík, R. Pěnička, V. Vonásek, and M. Saska, "Model predictive path integral control for agile unmanned aerial vehicles," in *IROS*, 2024.
- [13] I. S. Mohamed, G. Allibert, and P. Martinet, "Model predictive path integral control framework for partially observable navigation: A quadrotor case study," in *2020 16th International Conference on Control, Automation, Robotics and Vision (ICARCV)*, 2020.
- [14] D. Falanga, P. Foehn, P. Lu, and D. Scaramuzza, "PAMPC: Perception-aware model predictive control for quadrotors," in *IROS*, 2018.
- [15] M. Sarvaiya, G. Li, and G. Loianno, "Hpa-mpc: Hybrid perception-aware nonlinear model predictive control for quadrotors with suspended loads," *IEEE Robotics and Automation Letters*, 2025.
- [16] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in *IEEE International Conference on Robotics and Automation*, 2011, pp. 2520–2525.
- [17] B. Zhou, J. Pan, F. Gao, and S. Shen, "Raptor: Robust and perception-aware trajectory replanning for quadrotor fast flight," *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1992–2009, 2021.
- [18] Q. Yu, C. Qin, L. Luo, H. H.-T. Liu, and S. Hu, "CPA-planner: Motion planner with complete perception awareness for sensing-limited quadrotors," *IEEE Robotics and Automation Letters*, 2022.
- [19] T. Marcucci, M. Petersen, D. von Wrangel, and R. Tedrake, "Motion planning around obstacles with convex optimization," *Science Robotics*, 2023.
- [20] Y. Gao, F. Messerer, N. v. Duijkeren, B. Houska, and M. Diehl, "Real-time-feasible collision-free motion planning for ellipsoidal objects," in *2024 IEEE 63rd Conference on Decision and Control (CDC)*, 2024.
- [21] R. Reiter, K. Baumgärtner, R. Quirynen, and M. Diehl, "Progressive smoothing for motion planning in real-time NMPC," in *European Control Conference (ECC)*, 2024, pp. 1816–1823.
- [22] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, "EGO-planner: An ESDF-free gradient-based local planner for quadrotors," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 478–485, 2021.
- [23] I. Yadav, M. Sebok, and H. G. Tanner, "Receding horizon navigation and target tracking for aerial detection of transient radioactivity," *The International Journal of Robotics Research*, 2023.
- [24] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, "Learning high-speed flight in the wild," *Science Robotics*, vol. 6, no. 59, p. eabg5810, 2021.
- [25] A. Bhattacharya, N. Rao, D. Parikh, P. Kunapuli, Y. Wu, Y. Tao, N. Matni, and V. Kumar, "Vision transformers for end-to-end vision-based quadrotor obstacle avoidance," in *IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025.
- [26] K. Lee, J. Gibson, and E. A. Theodorou, "Aggressive perception-aware navigation using deep optical flow dynamics and pixelmpc," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1207–1214, 2020.
- [27] Z. Zhang and D. Scaramuzza, "Fisher information field: an efficient and differentiable map for perception-aware planning," 2020. [Online]. Available: <https://arxiv.org/abs/2008.03324>
- [28] J. Lim, N. Lawrance, F. Achermann, T. Stastny, R. Girod, and R. Siegwart, "Fisher information based active planning for aerial photogrammetry," in *ICRA*. IEEE, 2023.
- [29] E. A. Theodorou and E. Todorov, "Relative entropy and free energy dualities: Connections to Path Integral and KL control," in *IEEE 51st IEEE Conference on Decision and Control (CDC)*, Dec. 2012.
- [30] H. Xue, C. Pan, Z. Yi, G. Qu, and G. Shi, "Full-order sampling-based mpc for torque-level locomotion control via diffusion-style annealing," in *ICRA*, 2025.
- [31] Y. Gao, F. Messerer, N. van Duijkeren, R. Dabir, and M. Diehl, "Semi-infinite programming for collision-avoidance in optimal and model predictive control," 2025. [Online]. Available: <https://arxiv.org/abs/2508.12335>
- [32] H. J. Suh, M. Simchowicz, K. Zhang, and R. Tedrake, "Do differentiable simulators give better policy gradients?" in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, Eds., vol. 162. PMLR, 17–23 Jul 2022, pp. 20668–20696.
- [33] J. L. Piovesan and H. G. Tanner, "Randomized model predictive control for robot navigation," in *IEEE International Conference on Robotics and Automation*. IEEE, 2009, pp. 94–99.
- [34] R. Y. Rubinstein, "Optimization of computer simulation models with rare events," *European Journal of Operational Research*, 1997.
- [35] K. Chua, R. Calandra, R. McAllister, and S. Levine, "Deep reinforcement learning in a handful of trials using probabilistic dynamics models," in *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, ser. NIPS'18. Red Hook, NY, USA: Curran Associates Inc., 2018, p. 4759–4770.
- [36] H. J. Kappen, "Linear Theory for Control of Nonlinear Stochastic Systems," *Physical Review Letters*, Nov. 2005.
- [37] E. A. Theodorou, "Nonlinear Stochastic Control and Information Theoretic Dualities: Connections, Interdependencies and Thermodynamic Interpretations," *Entropy*, no. 5, May 2015.
- [38] G. Williams, A. Aldrich, and E. A. Theodorou, "Model predictive path integral control: From theory to parallel computation," *Journal of Guidance, Control, and Dynamics*, vol. 40, no. 2, pp. 344–357, 2017.
- [39] J. Alvarez-Padilla, J. Z. Zhang, S. Kwok, J. M. Dolan, and Z. Manchester, "Real-time whole-body control of legged robots with model-predictive path integral control," in *ICRA*, 2025.
- [40] R. Reiter, J. Hoffmann, D. Reinhardt, F. Messerer, K. Baumgärtner, S. Sawant, J. Boedecker, M. Diehl, and S. Gros, "Synthesis of model predictive control and reinforcement learning: Survey and classification," 2025. [Online]. Available: <https://arxiv.org/abs/2502.02133>
- [41] H. Homburger, F. Messerer, M. Diehl, and J. Reuter, "Optimality and suboptimality of MPPI control in stochastic and deterministic settings," *IEEE Control Systems Letters*, vol. 9, pp. 763–768, 2025.
- [42] Y. Ren, Y. Cai, F. Zhu, S. Liang, and F. Zhang, "ROG-Map: An efficient robocentric occupancy grid map for large-scene and high-resolution lidar-based motion planning," in *IROS*, 2024.
- [43] T. Howell, N. Gileadi, S. Tunyasuvunakool, K. Zakka, T. Erez, and Y. Tassa, "Predictive sampling: Real-time behaviour synthesis with mujoco," 2022. [Online]. Available: <https://arxiv.org/abs/2212.00541>
- [44] J. Amanatides and A. Woo, "A fast voxel traversal algorithm for ray tracing," *Proceedings of EuroGraphics*, vol. 87, 08 1987.
- [45] P. Foehn, E. Kaufmann, A. Romero, R. Penicka, S. Sun, L. Bauersfeld, T. Laengle, G. Cioffi, Y. Song, A. Loquercio, and D. Scaramuzza, "Agilicious: Open-source and open-hardware agile quadrotor for vision-based flight," *Science Robotics*, vol. 7, no. 67, p. eabl6259, 2022.
- [46] Y. Song, S. Naji, E. Kaufmann, A. Loquercio, and D. Scaramuzza, "Flightmare: A flexible quadrotor simulator," in *CoRL*, 2020.
- [47] S. K. Ramakrishnan, A. Gokaslan, E. Wijmans, O. Maksymets, A. Clegg, J. M. Turner, E. Undersander, W. Galuba, A. Westbury, A. X. Chang, M. Savva, Y. Zhao, and D. Batra, "Habitat-matterport 3d dataset (HM3d): 1000 large-scale 3d environments for embodied AI," in *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.

- [48] M. Dharmadhikari, P. De Petris, H. Nguyen, M. Kulkarni, N. Khedekar, and K. Alexis, “Manhole detection and traversal for exploration of ballast water tanks using micro aerial vehicles,” in *International Conference on Unmanned Aircraft Systems (ICUAS)*, 2023, pp. 103–109.